Network & Port Scanner Design Report

A structured explanation of the architecture, libraries, and network observations behind the Python scanning tool.

Author: Max/Mahmoud/Muza
Course: Cybersecurity / Networking Project

1. Introduction

This report explains the design and structure of the Python Network & Port Scanner developed during the project. The goal of the scanner was to explore how security tools detect open services on a network by sending packets (Tcp specifically)and analyzing responses from target hosts.

The report focuses on three main areas: code structure, libraries used , and observations made with Wireshark before building the scanner.

2. Code Structure and Design

2.1 Modular Function Organization

The scanner was designed using modular functions so that each part of the code has a clear responsibility. This approach improves readability, debugging, and maintainability.

The main components of the code include:

Component	Purpose
Import Section	Loads required Python libraries used for networking, file management, timing, and threading.
Scan File Management	Handles saved scan results and ensures previous results are not overwritten.
TCP Scan Module	Attempts TCP connections to determine if ports are open or closed.
UDP Scan Module	Sends UDP probes and analyzes responses to determine port status.
Threaded Scan Runner	Uses multiple threads to scan many ports simultaneously.
Result Display System	Formats and prints results based on user-selected display modes.
Menu Interface	Provides an interactive interface for users to run scans or view saved results.
Reconnaissance Tools	Includes ping, DNS lookup, and ARP table viewing tools.

2.2 Threading for Performance

The scanner uses Python's ThreadPoolExecutor to run multiple port scans at the same time. Network scanning involves waiting for responses from remote hosts, so scanning sequentially would be slow. By running many scans concurrently, the program significantly reduces total scan time.

2.3 Design

1. Import section

At the top, the code imports Python modules like:

- socket
- time
- os
- re
- ThreadPoolExecutor

```
import socket
import time
import os
import re
from concurrent.futures import ThreadPoolExecutor, as_completed
from udp_probes import get_probe, get_probe_service
```

These are tools Python gives us.

They help the program:

- talk to other computers on the network
- measure time
- read and write files
- search for saved scan files
- run many scan tasks at the same time

---

2. Managing saved result files

The functions:

- get_scan_files()

```python
#==============================================================
# All the functions created in this section from line 15-287
#-----------------------------
#Sorts the filename order to show 1 as the top of the list
#-----------------------------

def get_scan_files():
    """
    Finds saved scan files and returns them sorted by numeric filename.
    """
    numbered_files = []
    for filename in os.listdir("."):
        match = re.match(SCAN_RESULTS_PATTERN, filename)
        if match:
            numbered_files.append((int(match.group(1)), filename))
    return sorted(numbered_files, key=lambda x: x[0])
```

- get_scanned_targets()

```python
#----------------------------
#
#----------------------------

def get_scanned_targets():
    """
    Maps previously scanned targets to their saved file numbers from scan file headers.
    """
    scanned_targets = {}
    for file_number, filename in get_scan_files():
        with (variable) first_line: str
            first_line = f.readline().strip()
        if first_line.startswith("Scan results for host:"):
            scanned_ip = first_line.split(":", 1)[1].strip()
            scanned_targets[scanned_ip] = file_number
    return scanned_targets
```

- get_next_scan_file_number()

```python
#----------------------------
#This counts what should the file number be
#----------------------------

def get_next_scan_file_number():
    """
    Returns the next sequential scan file number based on existing saved scan files.
    """
    scan_files = get_scan_files()
    if not scan_files:
        return 1
    return scan_files[-1][0] + 1
```

- get_next_available_scan_filename()

```python
# ----------------------------
# Searches for and sets the nmming concvention for the files
# ----------------------------

def get_next_available_scan_filename():
    """
    Finds the next available scan filename without overwriting existing result files.
    """
    file_number = get_next_scan_file_number()
    filename = f"scan_results_{file_number}.txt"
    while os.path.exists(filename):
        file_number += 1
        filename = f"scan_results_{file_number}.txt"
    return file_number, filename
```

are used to manage saved scan results.

What they do

They make sure that when a scan is saved, the program creates files like:

- scan_results_1.txt
- scan_results_2.txt
- scan_results_3.txt

instead of overwriting old results.

This is useful because it keeps a history of previous scans. We could have added date and time when scans were implemented but decided not to go forward with it.

3. Saving scan results

The function:

save_scan_results(target_ip, results, scan_type, ports)

```
def save_scan_results(target_ip, results, scan_type, ports):
    Saves current scan results with target details to a new numbered text file.
    """
    _, filename = get_next_available_scan_filename()
    start = time.time()

    if not ports:
        port_summary = "None"
    elif len(ports) == 1:
        port_summary = str(ports[0])
    else:
        min_port = min(ports)
        max_port = max(ports)
        if ports == list(range(min_port, max_port + 1)):
            port_summary = f"{min_port}-{max_port}"
        else:
            port_summary = ", ".join(str(port) for port in ports)

    with open(filename, "w") as f:
        f.write(f"Scan results for host: {target_ip}\n")
        f.write(f"Scan type: {scan_type}\n")
        f.write(f"Ports: {port_summary}\n")
        f.write("\nPort status:\n")
        f.write(f"{'PORT':<8}{'PROTO':<8}{'STATE':<15}{'SERVICE'}\n")
        f.write("-" * 50 + "\n")
        for port, proto, state in results:
            service = ""
            try:
                service = socket.getservbyport(port, proto)
            except:
                service = get_probe_service(port) or ""
            f.write(f"{port:<8}{proto:<8}{state:<15}{service}\n")
```

takes the scan output and writes it into a file.

What it includes

It saves:

- the target IP address
- the type of scan

- the ports checked
- whether each port was open, closed, or filtered
- the likely service name for the port

This helps make the project more practical, because users can review results later instead of only seeing them on screen once.

---

4. TCP scanning

The function:

scan_tcp(ip, port)

```
# ----------------------------
# TCP scan
# ----------------------------

def scan_tcp(ip, port):
    """
    Scans one TCP port and returns a standardized result containing port, protocol, and state.
    """
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(2)
        result = sock.connect_ex((ip, port))
        sock.close()
        if result == 0:
            return (port, "tcp", "open")
        else:
            return (port, "tcp", "closed")
    except:
        return (port, "tcp", "filtered")
```

checks one **TCP** port.

What it does in simple language

It tries to connect to a port.

- If the connection works, the program says the port is **open**
- If it does not work, it says **closed**
- If something unusual happens, it may label it **filtered**

This is a very straightforward way to test TCP ports.

---

5. UDP scanning

The function:

scan_udp(ip, port)

```python
def scan_udp(ip, port):
    Scans one UDP port with retries and returns a standardized port state result.
    """

    retries = 2
    timeout = 1

    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.settimeout(timeout)

        probe = get_probe(port)

        for _ in range(retries):

            sock.sendto(probe, (ip, port))

            try:
                sock.recvfrom(1024)
                sock.close()
                return (port, "udp", "open")

            except socket.timeout:
                continue

        sock.close()
        return (port, "udp", "filtered")

    except ConnectionRefusedError:
        return (port, "udp", "closed")

    except:
        return (port, "udp", "filtered")
```

checks one **UDP** port.

UDP is harder to scan than TCP because UDP often gives **less clear answers**.

What the code does

It sends a small test message, called a **probe**, to the target port and waits for a reply.

- If a reply comes back, the port is treated as **open**
- If no reply comes after retries, it is marked **filtered**
- If the target rejects it, it may be marked **closed**

This is why UDP scanning is often slower and less certain than TCP scanning.

---

6. Running many scans at once with threading

The function:

run_scan(ip, ports, scan_type)

```python
# ----------------------------
# Threaded scan runner
# ----------------------------

def run_scan(ip, ports, scan_type):
    """
    Runs selected scan types concurrently across ports and returns sorted scan results.
    """
    results = []
    with ThreadPoolExecutor(max_workers=100) as executor:
        futures = []
        for port in ports:
            if scan_type in ("tcp", "tcp+udp"):
                futures.append(executor.submit(scan_tcp, ip, port))
            if scan_type in ("udp", "tcp+udp"):
                futures.append(executor.submit(scan_udp, ip, port))
        for future in as_completed(futures):
            results.append(future.result())
    return sorted(results, key=lambda x: (x[0], x[1]))
```

is where the threading happens.

It uses:

ThreadPoolExecutor(max_workers=100)

This means the program can work on many ports at the same time instead of checking one by one.

---

7. Why we used threading

This is one of the most important design choices in the program.

Example without threading(our first initial program just reading sockets or ports one at a time):

```
Scan results for host: 192.168.56.103
Port status:
port 0 is closed
port 1 is closed
port 2 is closed
port 3 is closed
port 4 is closed
port 5 is closed
port 6 is closed
port 7 is closed
port 8 is closed
port 9 is closed
port 10 is closed
port 11 is closed
port 12 is closed
port 13 is closed
port 14 is closed
port 15 is closed
port 16 is closed
port 17 is closed
port 18 is closed
port 19 is closed
port 20 is closed
port 21 is open
port 22 is closed
port 23 is closed
```

It took too long and not accurate at all.

Without threading

The scanner would do this:

1. check port 1
2. wait
3. check port 2
4. wait
5. check port 3
6. wait

That would be slow, especially if scanning many ports.

With threading

The scanner can check many ports at the same time.

So instead of waiting for each one to finish before starting the next, it can launch many scan tasks together.

Why this matters

Port scans often spend a lot of time **waiting for network responses**. Since waiting is the main delay, threading helps reduce total scan time a lot.

---

8. Why threading was useful in our project

We used threading because:

- it made the scanner faster
- it let us scan a range of ports more efficiently
- it made the program feel closer to a real security tool
- it reduced waiting time for the user

In other words, threading improved the **performance** and **usability** of the scanner.

---

9. Display options in the code

The program also lets the user choose how much detail to show.

```python
# ----------------------------
# Display results
# ----------------------------

def display_results(results, display_mode):
    """
    Prints scan results in a table and filters output using the chosen display mode.
    """
    print("\nPORT     PROTO   STATE           SERVICE")
    print("---------------------------------------------")
    for port, proto, state in results:
        # Mode 1: open + open|filtered
        if display_mode == 1 and state == "closed":
            continue
        # Mode 3: only open
        if display_mode == 3 and state != "open":
            continue

        service = ""
        try:
            service = socket.getservbyport(port, proto)
        except:
            service = ""

        print(f"{port:<8}{proto:<8}{state:<15}{service}")
```

The display_results() function can:

- show only open ports
- show open and filtered ports
- show open, filtered, and closed ports

Why that is useful

If a user scans many ports, showing every closed port can make the output messy.

Filtering the display makes the results easier to read.

---

10. Menu system and user interaction

The code includes a menu so the user can choose from different options:

```
play = True
while play == True:
    banner = r"""
    [1] Start Scan
    [2] Export to file results
    [3] Ping target and os comands for reconnaissance
    [4] Quit
    """
    print(banner)
    print("Main Menu :")
    print("="*90)
    answer = input(f"1. Start scan \n2. Export to file results \n3. Ping target and os commands for reconnaissance \n
```

- start scan
- view saved scan files
- use simple reconnaissance commands
- quit

This makes the program easier to use because it behaves more like a small application instead of just a single script.

---

10. Extra reconnaissance tools

The program also includes:

- ping_host()
- dns_lookup()
- show_arp()

```python
# ----------------------------
# Ping and OS commands for reconnaissance
# ----------------------------

def ping_host():
    target = input("Enter IP or hostname to ping:")
    if os.name =="nt":
        os.system(f"ping {target}")
    else:
        os.system(f"ping -c 4 {target}")

def dns_lookup():
    domain = input("Enter domain name to look up:")
    os.system(f"nslookup {domain}")

def show_arp():
    os.system("arp -a")
```

These are simple support tools.

Why they matter

Before scanning, a user may want to:

- check whether a host is reachable
- look up a domain name
- view nearby network devices in the ARP table

These are small but useful additions that make the scanner more complete.

3. Libraries Used

Library	Purpose
socket	Creates TCP and UDP connections used to test whether ports are open.
time	Used to measure scan timing and execution duration.
os	Allows interaction with the operating system for file management and system commands.
re	Used for pattern matching when managing scan result filenames.
concurrent.futures	Provides threading tools that allow many scan tasks to run simultaneously.
udp_probes	Provides probe payloads and service detection hints for UDP scanning.

4. Wireshark Observations Before Implementation

Before writing the scanner, packet captures were analyzed using Wireshark to understand how port scanning works at the network level.

4.1 TCP Behavior

Wireshark demonstrated how TCP connections behave when a port is open or closed. When a scanner attempts to connect to a port, the host responds differently depending on the service status. These response patterns allow the scanner to determine whether a port is open, closed, or filtered.

4.2 UDP Behavior

UDP scanning is less reliable because UDP does not use a connection handshake. In many cases, a service may not respond at all. Observing UDP traffic in Wireshark helped demonstrate why scanners sometimes classify ports as 'open|filtered'.

4.3 Understanding Scanner Logic

The Wireshark analysis reinforced the idea that scanning tools simply send packets and interpret responses. The logic implemented in the scanner follows this same principle: generate traffic, wait for a reply, and classify the port based on the response or lack of response.

5. Project Strengths

The final scanner includes several useful features:

- Support for both TCP and UDP scanning

- Threaded scanning for improved performance
- Saving scan results to numbered files
- Interactive menu system
- Reconnaissance tools such as ping and DNS lookup
- Configurable display modes for scan output

6. Conclusion

Building this scanner provided practical insight into how network security tools operate. By analyzing packet behavior in Wireshark and implementing scanning logic in Python, the project demonstrated the relationship between network traffic, protocol behavior, and Python.